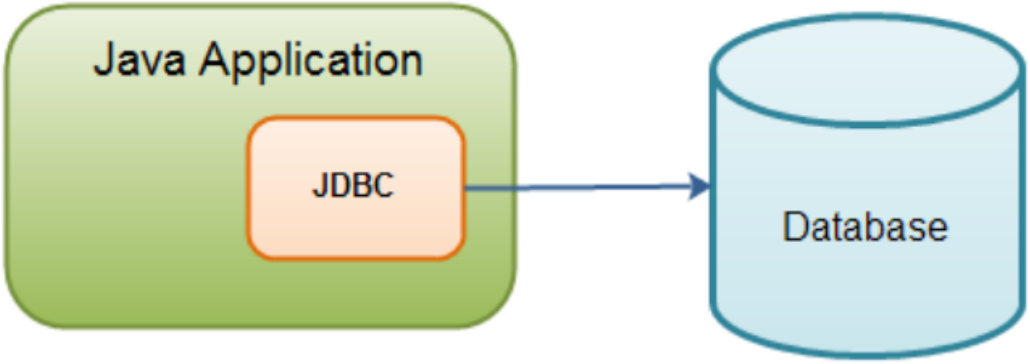
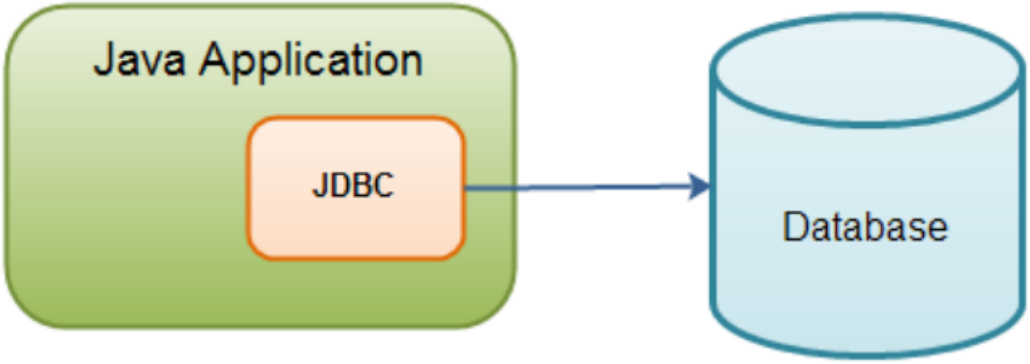

JDBC
Java Database Connection

Una de las principales aplicaciones de cualquier lenguaje moderno es la posibilidad de utilizar datos pertenecientes a un sistema de base de datos. La dificultad del manejo de archivos y las facilidades de manejo de datos que ofrecen los sistemas gestores de base de datos (SGBDs) son los causantes de esta necesidad.





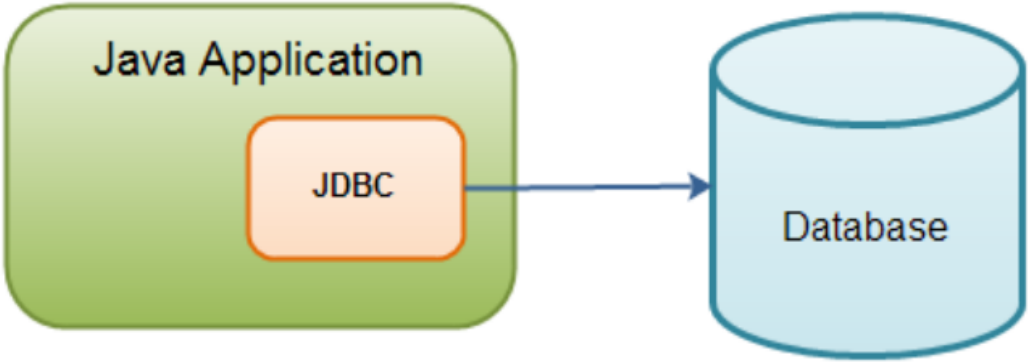
La API JDBC de Java (Java Database Connectivity) permite que las aplicaciones Java se conecten a bases de datos relacionales como MySQL, PostgreSQL, MS SQL Server, Oracle, H2 Database, etc. La API JDBC permite consultar y actualizar bases de datos relacionales. La API JDBC de Java es parte del SDK de Java SE, lo que hace que JDBC esté disponible para todas las aplicaciones Java que quieran usarlo.



JDBC es independiente de la base de datos

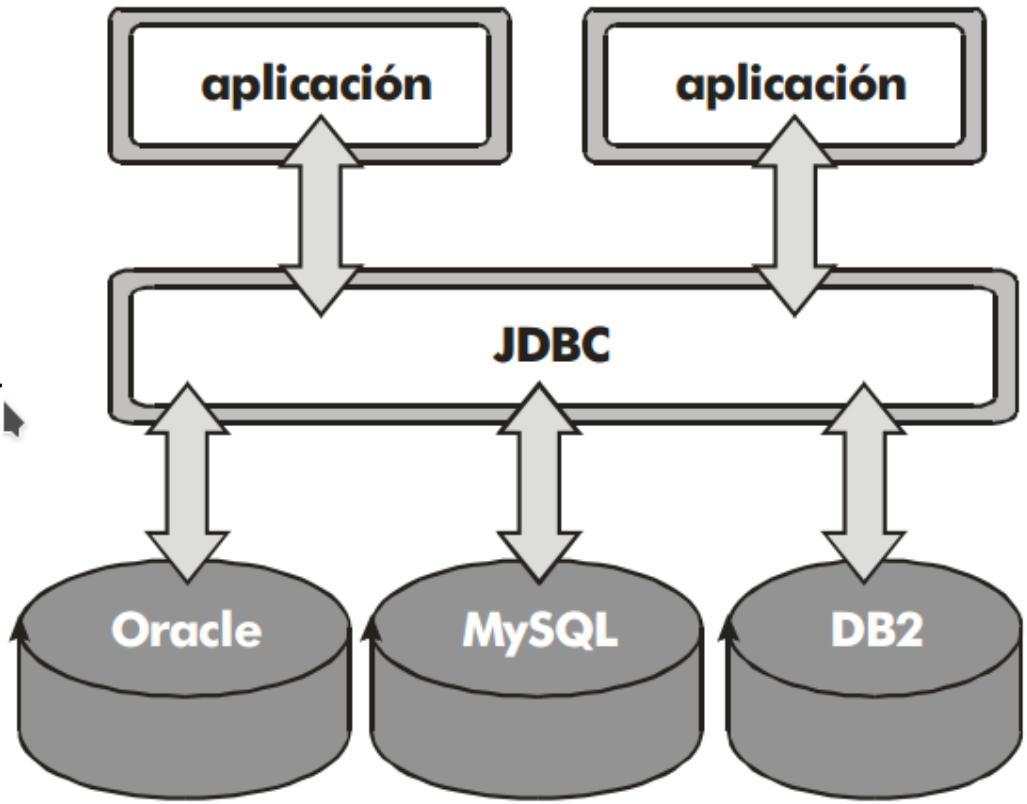
JDBC no es independiente de SQL

JDBC no es para bases de datos no relacionales



Estructura JDBC

Las aplicaciones sólo se comunican con la interfaz JDBC. Ésta es el encargada de comunicarse con los sistemas de base de datos.
Es necesario adquirir un controlador JDBC para el sistema gestor de base de datos que utilicemos. La comunicación fundamental entre las aplicaciones y JDBC se realiza a través de instrucciones SQL.



Empezar con JDBC jdbc.doc

Instalar Java y el JDBC en nuestra máquina

Instalar un driver en nuestra máquina.

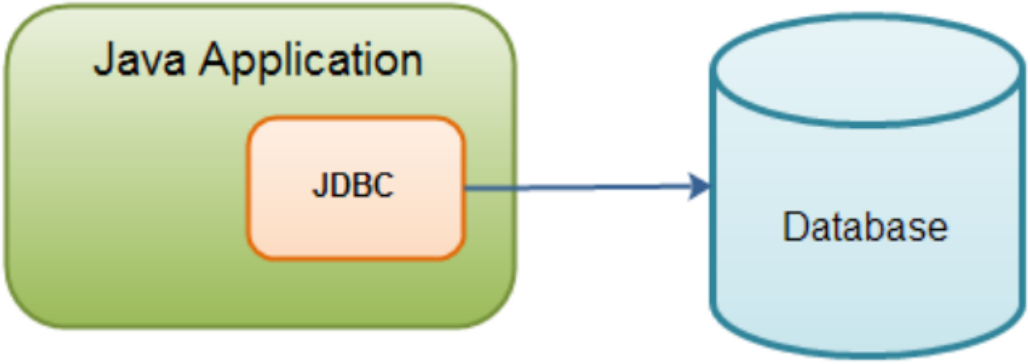
Instalar nuestro Controlador de Base de Datos si es necesario.

Usar JDBC

El código Java debe incluir ciertas instrucciones para establecer una conexión y comunicación con una base de datos a través de JDBC.

Dichas instrucciones deben considerar los siguientes aspectos:

1) Cargar los Drivers
2) Hacer una conexión
3) Crear sentencias JDBC
4) Ejecutar sentencias
5) Cerrar conexión



EJEMPLO

Parte 1



1) Cargar los Drivers
2) Hacer una conexión
3) Crear sentencias JDBC
4) Ejecutar sentencias
    1) Crear una tabla
    2) Insertar filas
5) Cerrar conexión

1) Se verá en la segunda parte

EJEMPLO
Parte 1




```
import java.sql.*;

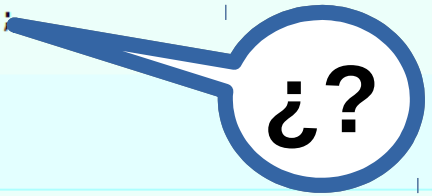
class java2101
{
  static public void main( String[] args )
  {
    Connection conexion;
    Statement sentencia;
    ResultSet resultado;

    System.out.println( "Iniciando programa." );

    // Se carga el driver JDBC-ODBC
    try
    {
      Class.forName( "sun.jdbc.odbc.JdbcOdbcDriver" );
    }
    catch( Exception e )
    {
      System.out.println( "No se pudo cargar el puente JDBC-ODBC." );
      return;
    }
```

```java
public void crearTabla()
{
    try
    {
        // Se establece la conexión con la base de datos
        String baseDatos = "jdbc:mysql://localhost:3306/Tutorial";
        String usuario = "leonel";
        String password = "leonel";
        conexion = DriverManager.getConnection(baseDatos, usuario, password);
        sentencia = conexion.createStatement();
        try
        {
            // Se elimina la tabla en caso de que ya existiese.
            // Esto es permitido para ejecutarlo como ejemplo.
            sentencia.execute("DROP TABLE AGENDA");
        }
        catch (SQLException e)
        {
        }

        // Esto es código SQL
        sentencia.execute("CREATE TABLE AMIGOS (" + " NOMBRE VARCHAR(15) NOT NULL, " +
                          " APELLIDOS VARCHAR(30) NOT NULL, " + " EDAD TINYINT) ");
    }
    catch (Exception e)
    {
        System.out.println(e);
        return;
    }
    System.out.println("Creacion finalizada.");
}
```



EJEMPLO
Parte 1



Las partes más interesantes del código son las que se van a revisar a continuación, profundizando en cada uno de los pasos.

Lo primero que se hace es importar toda la funcionalidad de JDBC, a través de la primera sentencia ejecutable del programa.

```
import java.sql.*;
```

Las siguientes líneas son las que cargan el puente JDBC-ODBC, mediante el método *forName()* de la clase **Class**.

```
try
{
    Class.forName( "sun.jdbc.odbc.JdbcOdbcDriver" );
}
catch( Exception e )
{
    System.out.println( "No se pudo cargar el puente JDBC-ODBC." );
    return;
}
```

El método ***forName()*** localiza, lee y enlaza dinámicamente una clase determinada. La sintaxis recomendada de ***forName()*** es *nombreEmpresa.nombreBaseDatos.nombreDriver*

El *driver* deberá estar ubicado en el directorio *nombreEmpresa\nombreBaseDatos\ nombreDriver.class* a partir del directorio indicado por la variable de entorno *CLASSPATH*.

Si por cualquier motivo no es posible conseguir cargar **JdbcOdbcDriver.class**, se intercepta la excepción y se sale del programa.

A continuación, se solicita a **DriverManager** que proporcione una conexión para una fuente de datos ODBC. El parámetro **jdbc:odbc:Tutorial** especifica que la intención es acceder a la fuente de datos con nombre **Tutorial**, *Data Source Name* o DSN, en la terminología ODBC.

```
try
{
  // Se establece la conexión con la base de datos
  conexion = DriverManager.getConnection( "jdbc:odbc:Tutorial","","" );
  sentencia = conexion.createStatement();
```

El segundo y tercer parámetro son el nombre del usuario y la clave con la cual se intentará la conexión. En este caso el acceso es libre

Inmediatamente después de obtener la conexión, en la siguiente línea

```
try
{
  // Se establece la conexión con la base de datos
  conexion = DriverManager.getConnection( "jdbc:odbc:Tutorial", "", "" );
  sentencia = conexion.createStatement();
```

se solicita que proporcione un objeto de tipo *Statement* para poder ejecutar sentencias a través de esa conexión. Para ello se dispone de los métodos *execute(String sentencia)* para ejecutar una petición SQL que no devuelve datos o *executeQuery(String sentencia)* para ejecutar una consulta SQL. Este último método devuelve un objeto de tipo *ResultSet*.

Una vez que se tiene el objeto *Statement* ya se pueden lanzar consultas y ejecutar sentencias contra el servidor. A partir de aquí el resto del programa realmente es SQL «adornado»: en la línea:

```
try
{
   // Se elimina la tabla en caso de que ya existiese
   sentencia.execute( "DROP TABLE AGENDA" );
}
catch( SQLException e ) {};
```



se ejecuta *DROP TABLE AMIGOS* para borrar cualquier tabla existente anteriormente. Puesto que este ejemplo es una aplicación «de instalación» y es posible que la tabla *AMIGOS* no exista, dando como resultado una excepción, se aísla la *sentencia.execute()* mediante un *try* y un *catch*.

```
// Esto es código SQL
sentencia.execute("CREATE TABLE AMIGOS (" + " NOMBRE VARCHAR(15) NOT NULL, " +
                  " APELLIDOS VARCHAR(30) NOT NULL, " + " EDAD TINYINT) ");
```

Se ejecuta una sentencia SQL que crea la tabla *AMIGOS* con tres campos: *NOMBRE, APELLIDOS* y *CUMPLE*. De ellos, únicamente el tercero, correspondiente al cumpleaños, es el que puede ser desconocido, es decir, puede contener valores nulos.

```java
public void insertarDatos()
{

    try
    {
        sentencia.execute("INSERT INTO AMIGOS " + "VALUES('JOSE','GONZALEZ',20)");
        sentencia.execute("INSERT INTO AMIGOS " + "VALUES('PEDRO','GOMEZ',30)");
        //sentencia.execute("INSERT INTO AMIGOS " + "VALUES('GONZALO','PEREZ', NULL)");
    }
    catch (Exception e)
    {
        System.out.println("Error al insertar datos");
    }
}
```

Se ejecutan sentencias *INSERT* para rellenar con datos la tabla. En todo momento se ha colocado un *try ... catch* exterior para interceptar cualquier excepción que puedan dar las sentencias. En general, para *java.sql* está definida una clase especial de excepciones que es ***SQLException***. Se obtendrá una excepción de este tipo cuando ocurra cualquier error de proceso de JDBC, tanto si es a nivel JDBC como si es a nivel inferior (ODBC o de protocolo).

```java
public void insertarDatos()
{

    try
    {
        sentencia.execute("INSERT INTO AMIGOS " + "VALUES('JOSE','GONZALEZ',20)");
        sentencia.execute("INSERT INTO AMIGOS " + "VALUES('PEDRO','GOMEZ',30)");
        //sentencia.execute("INSERT INTO AMIGOS " + "VALUES('GONZALO','PEREZ', NULL)");
    }
    catch (Exception e)
    {
        System.out.println("Error al insertar datos");
    }
}
```

Por ejemplo, si en lugar de *GONZALO* en la línea correspondiente a la última inserción en la Base de Datos, se intenta añadir un nombre nulo (*NULL*), se generará una excepción *SQLException* con el mensaje
[Microsoft][ODBC SQL Server Driver][SQL Server]Attempt to insert the value NULL into column 'NOMBRE', table 'master.dbo.AGENDA'; column does not allow nulls. INSERT fails.
que en el caso de *Microsoft Access* sería:
[Microsoft][ODBC Microsoft Access 97 Driver] The field 'AGENDA.NOMBRE' can't contain a Null value because the Required property for this field is set to True. Enter a value in this field.

EJEMPLO

Parte 2



1)Cargar los Drivers

2)Hacer una conexión

3)Crear sentencias JDBC

4)Ejecutar sentencias

   1) Obtener información a partir de una BD creada.

5)Cerrar conexión

EJEMPLO

Parte 2



Solamente se mostrarán las partes de código significativas

Pasos que hay que dar para obtener información a partir de una base de datos ya creada.

```
public void consultar()
{
    try
    {
        resultado = sentencia.executeQuery("SELECT * FROM AMIGOS");
        while (resultado.next())
        {
            String nombre = resultado.getString("NOMBRE");
            String apellidos = resultado.getString("APELLIDOS");
            int cumple = resultado.getInt("EDAD");
            System.out.println("El aniversario de D. " + nombre +
                               " " + apellidos + ", la edad es " + cumple);
        }
    }
    catch (Exception e)
    {
        System.out.println("errores al consultar");
    }
}
```

```java
public void consultar()
{
    try
    {
        resultado = sentencia.executeQuery("SELECT * FROM AMIGOS");
        while (resultado.next())
        {
            String nombre = resultado.getString("NOMBRE");
            String apellidos = resultado.getString("APELLIDOS");
            int cumple = resultado.getInt("EDAD");
            System.out.println("El aniversario de D. " + nombre +
                               " " + apellidos + ", la edad es " + cumple);
        }
    }
    catch (Exception e)
    {
        System.out.println("errores al consultar");
    }
}
```

Se utiliza *executeQuery* para obtener el resultado de *SELECT * FROM AMIGOS*. Mediante *resultado.next()* la posición se situará en el «siguiente» elemento del resultado, o bien sobre el primero si todavía no se ha utilizado. La función *next()* devuelve *true* o *false* si el elemento existe, de forma que se puede iterar mediante *while (resultado.next())* para tener acceso a todos los elementos.

```java
public void cerrarConexion()
{
    if (conexion != null)
    {
        try
        {
            sentencia.close();
            conexion.close();
        }
        catch (SQLException e)
        { /* ignored */
        }
    }
}
```

Cerrando las conexiones se garantiza que se actualice la información en la base de datos. Hasta este momento no hay seguridad de que esté actualizada.
Se va utilizando una memoria cache.